

ESTRUCTURA DE DATOS · UNIDAD 3

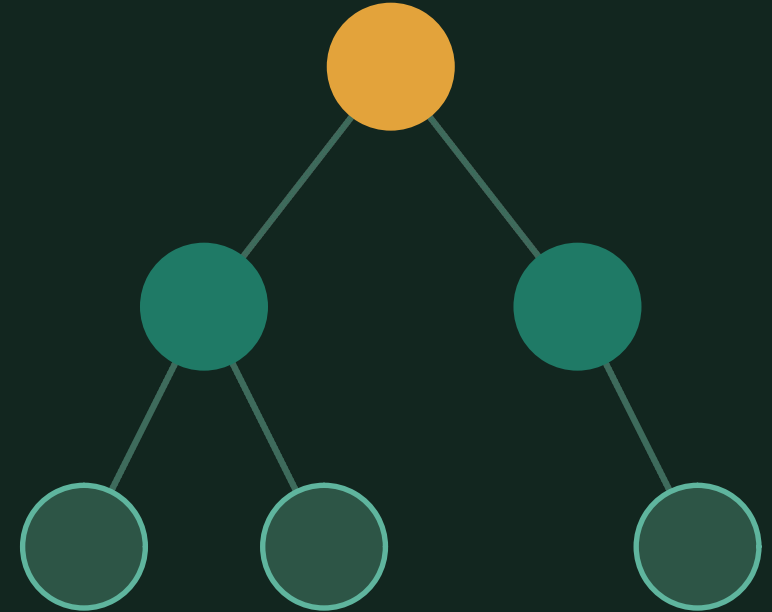
Introducción a los árboles

Terminología, árbol general y árbol binario, y la clase Nodo

Grupo 1 · Jueves 08/10/2026

Docente: José Rubén Caiza Caizabuano · UTA FISEI, Software, Nivel III

[Integrantes del Grupo 1]



EL PROBLEMA

La jerarquía en los datos

Como lista

Documentos

Universidad

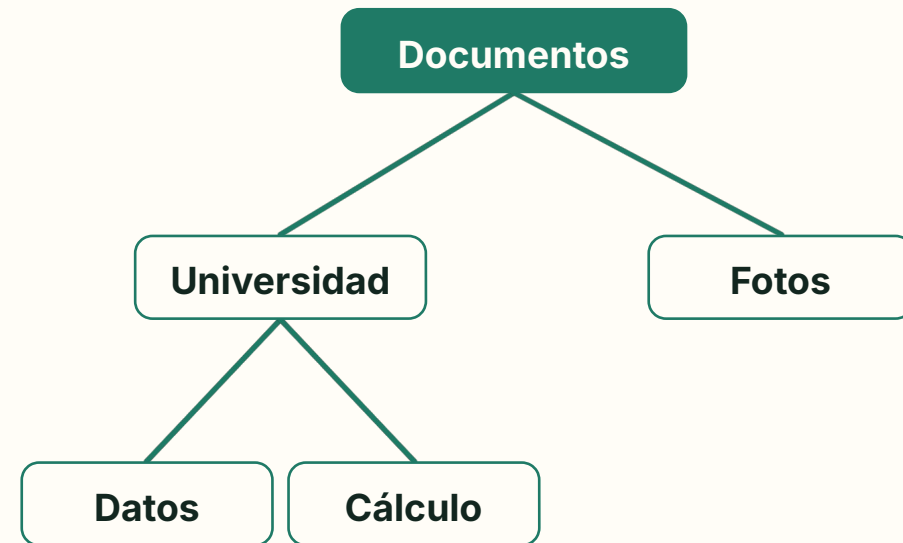
Datos

Cálculo

Fotos

No se ve qué carpeta está dentro de cuál.

Como árbol



Cada carpeta cuelga de una sola carpeta.

CONCEPTOS

¿Qué es un árbol?

Un árbol es una **estructura jerárquica**: tiene una raíz, y todo nodo, menos la raíz, tiene exactamente un padre.

Conexo

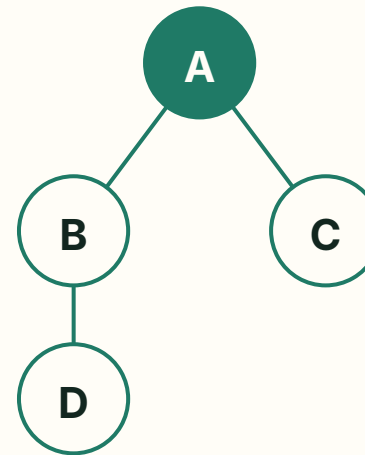
Desde la raíz se llega a todos los nodos.

Sin ciclos

De la raíz a cada nodo hay un solo camino.

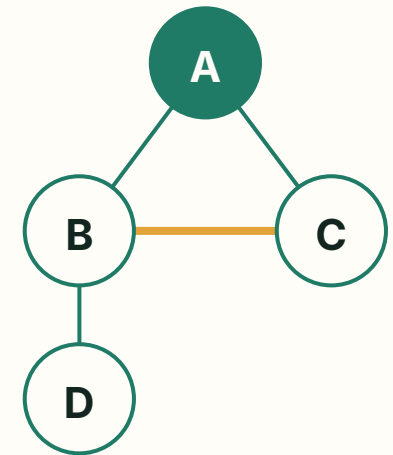
En grafos se dice vértice: nodo y vértice significan lo mismo.

Sí es árbol



De A a D hay un solo camino

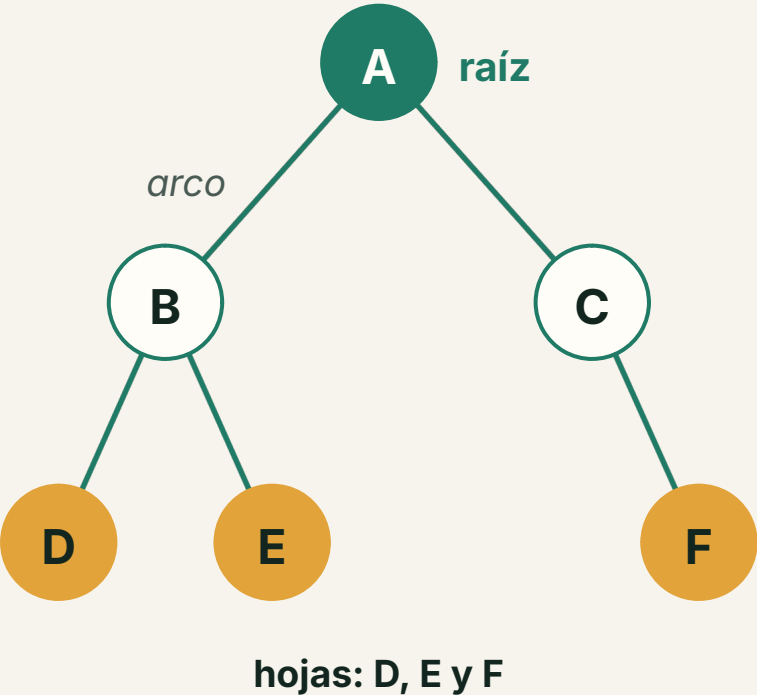
No es árbol



De A a C hay dos caminos:
A, C y A, B, C

CONCEPTOS

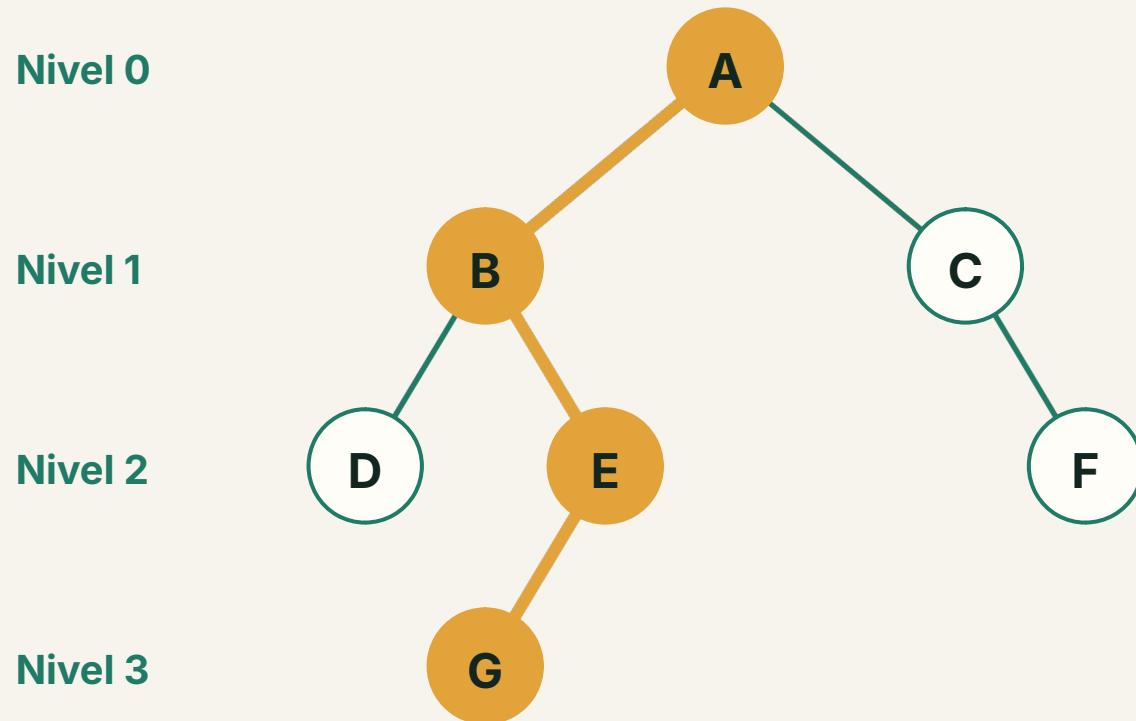
Terminología básica



Término	Significado
Raíz	Nodo sin padre (A)
Nodo	Elemento que guarda un dato
Arco	Enlace entre padre e hijo
Padre e hijo	B es hijo de A
Hoja	Nodo sin hijos (D, E, F)
Grado	Cantidad de hijos (A: 2)

CONCEPTOS

Nivel y altura



Nivel

Distancia de un nodo a la raíz. La raíz está en el nivel 0.

Altura

Camino más largo de la raíz a una hoja, contado en aristas.

Altura = 3

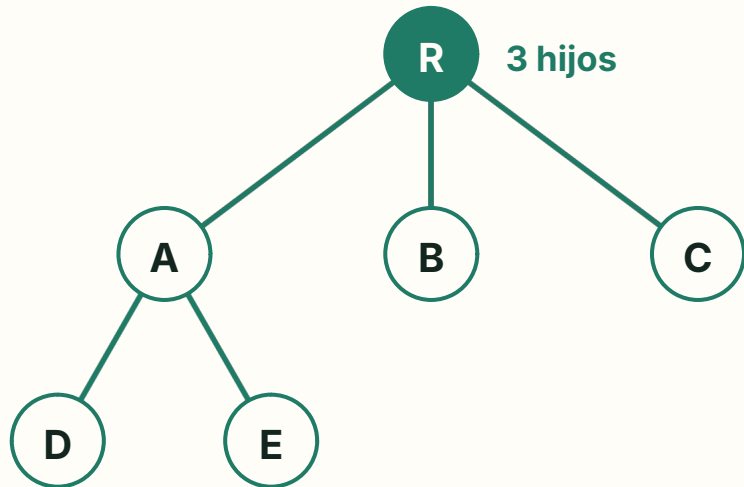
camino A, B, E, G

CONCEPTOS

Árbol general y árbol binario

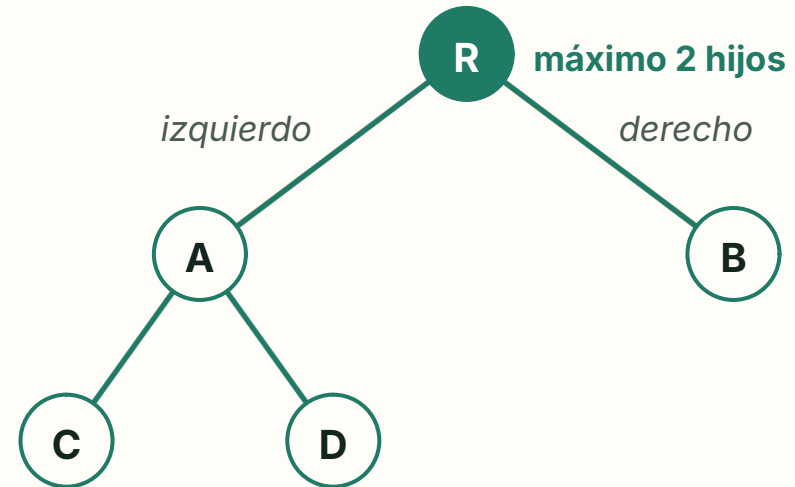
Árbol general

Cada nodo puede tener cualquier cantidad de hijos.



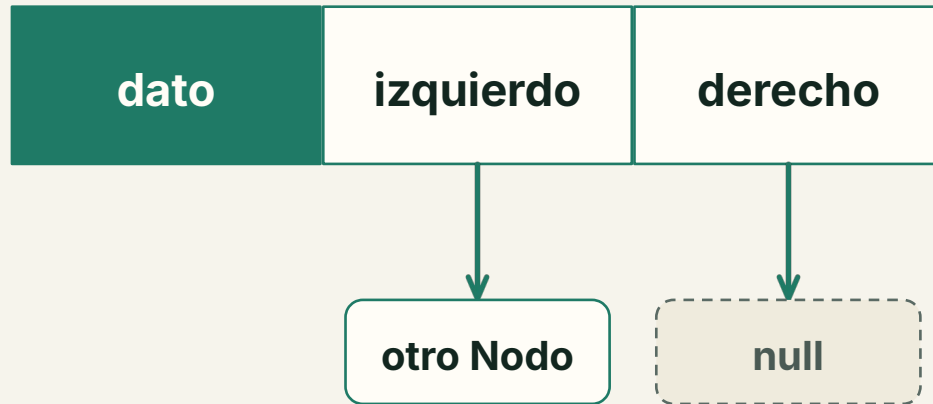
Árbol binario

Cada nodo tiene como máximo dos hijos: el izquierdo y el derecho.



EN JAVA

La clase Nodo



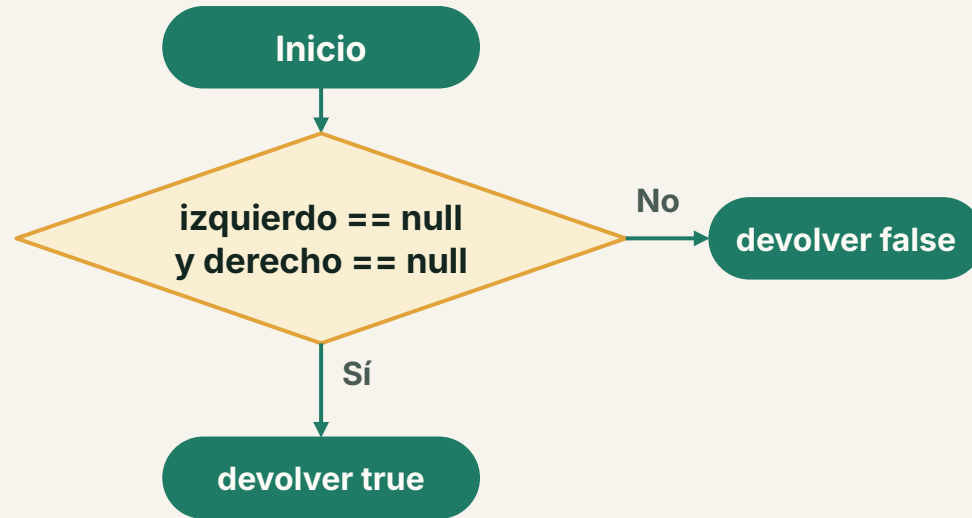
Si un enlace es null, ese hijo no existe. Una hoja es un nodo con los dos enlaces en null.

```
public class Nodo<T> {
    private T dato;
    private Nodo<T> izquierdo;
    private Nodo<T> derecho;
    public Nodo(T dato) { this.dato = dato; }
    public boolean esHoja() { ... }
}
```

OPERACIONES DEL MODELO

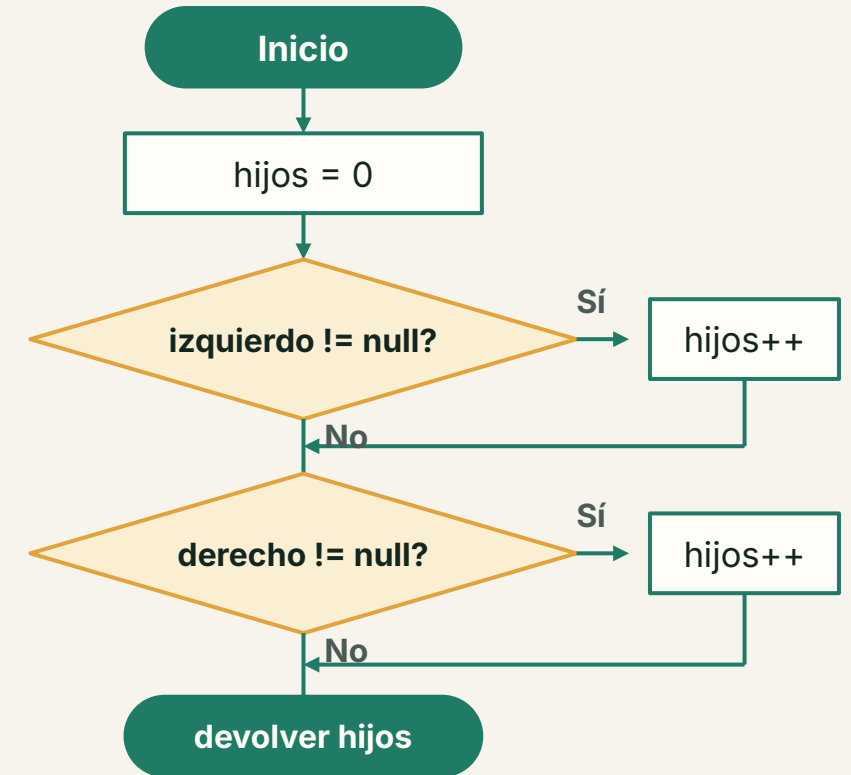
Nodo: esHoja() y grado()

`boolean esHoja()`



Una hoja es un nodo sin hijos: los dos enlaces valen null.

`int grado()`



OPERACIONES

Las operaciones de cada clase

MODELO · `Nodo<T>`

<code>getDato / setDato</code>	lee y cambia el dato
<code>getIzquierdo / setIzquierdo</code>	enlace izquierdo
<code>getDerecho / setDerecho</code>	enlace derecho
<code>esHoja()</code>	true si no tiene hijos
<code>grado()</code>	cantidad de hijos: 0, 1 o 2

LAS TRES CAPAS



Cada capa solo usa a la siguiente.

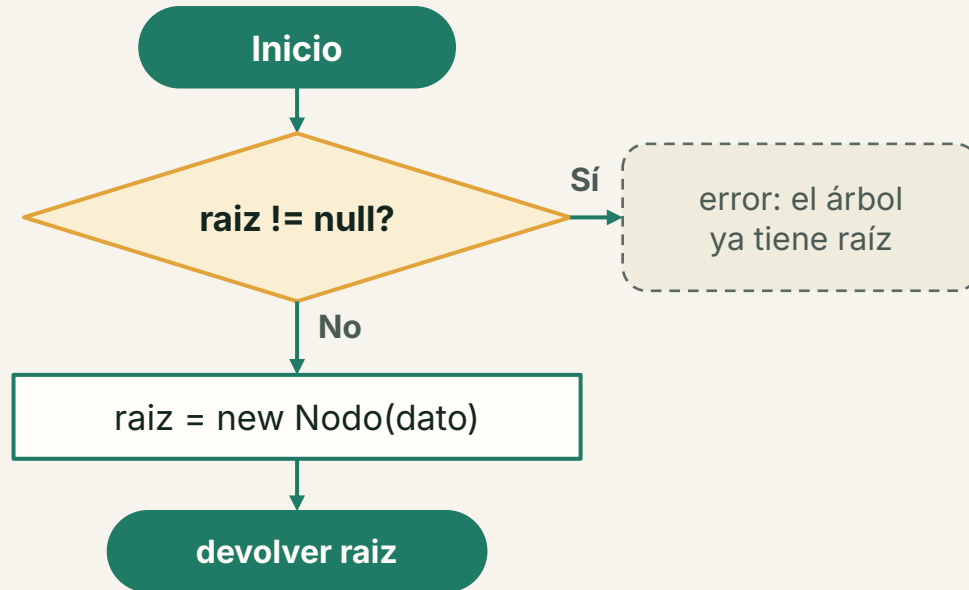
NEGOCIO · `ArbolBinario<T>`

<code>esVacio()</code>	true si no hay raíz
<code>getRaiz()</code>	devuelve la raíz
<code>crearRaiz(dato)</code>	pone la primera raíz
<code>agregarIzquierdo(padre, dato)</code>	cuelga un hijo nuevo
<code>agregarDerecho(padre, dato)</code>	igual, del lado derecho
<code>contarNodos()</code>	cuenta todos los nodos
<code>contarHojas()</code>	cuenta las hojas
<code>altura()</code>	camino más largo, en aristas

OPERACIONES DEL NEGOCIO

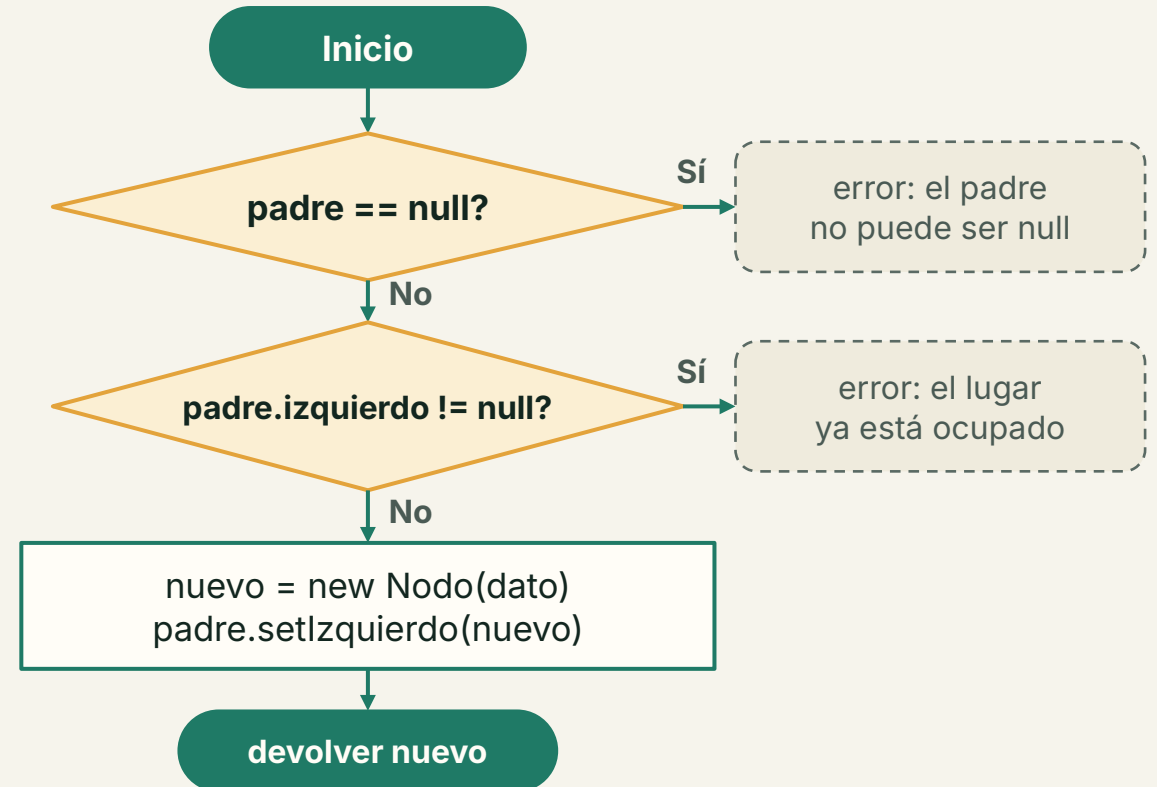
crearRaiz() y agregarIzquierdo()

Nodo<T> crearRaiz(T dato)



agregarDerecho() hace lo mismo con el lado derecho.

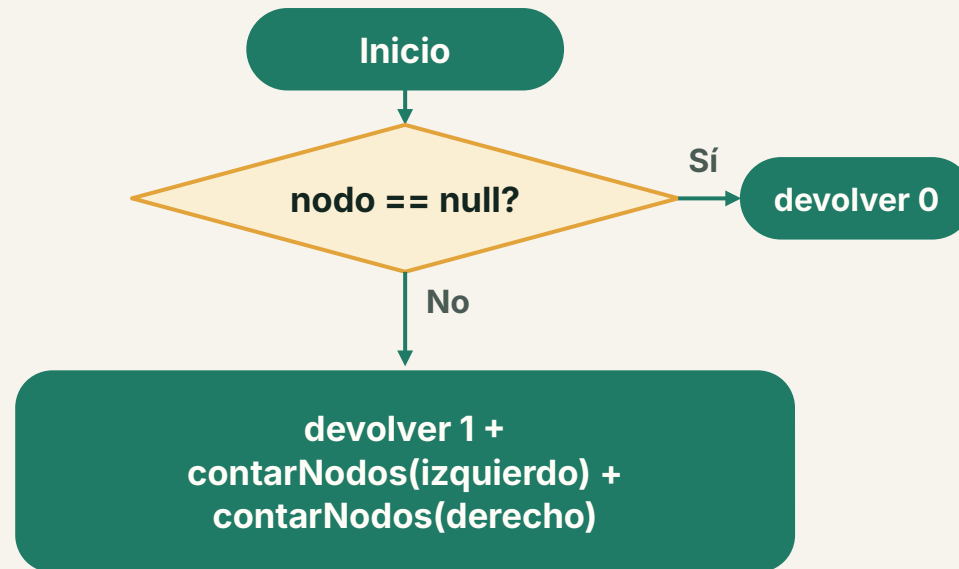
Nodo<T> agregarIzquierdo(padre, dato)



OPERACIONES DEL NEGOCIO

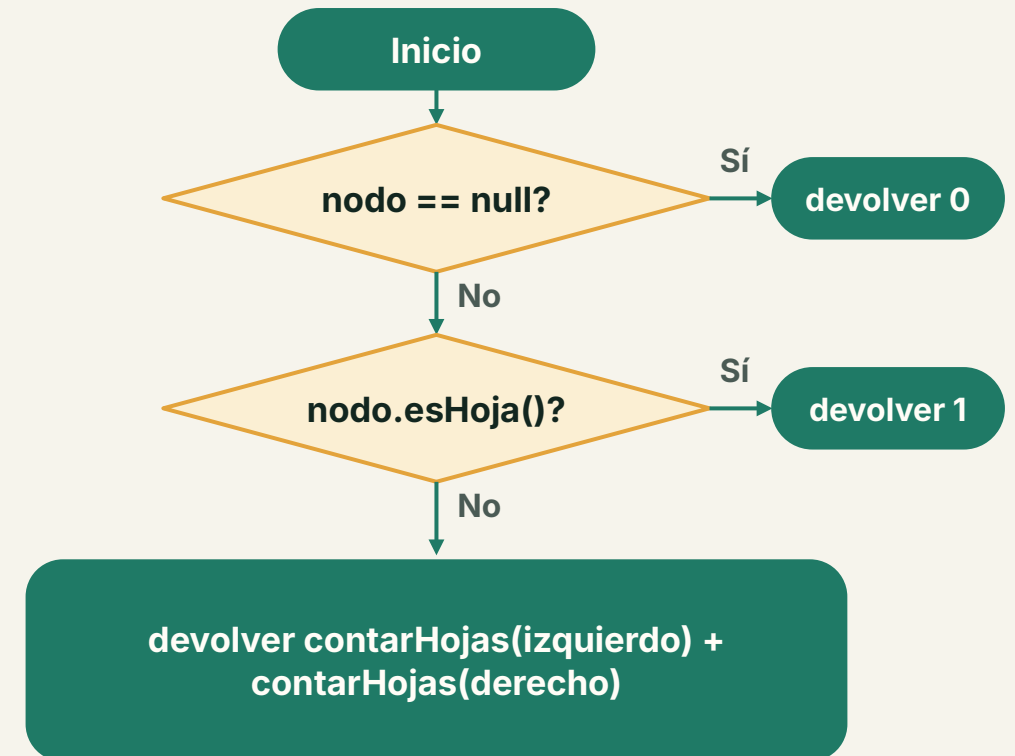
contarNodos() y contarHojas()

```
int contarNodos(Nodo<T> nodo)
```



Cada llamada resuelve un subárbol más pequeño, hasta llegar a null.

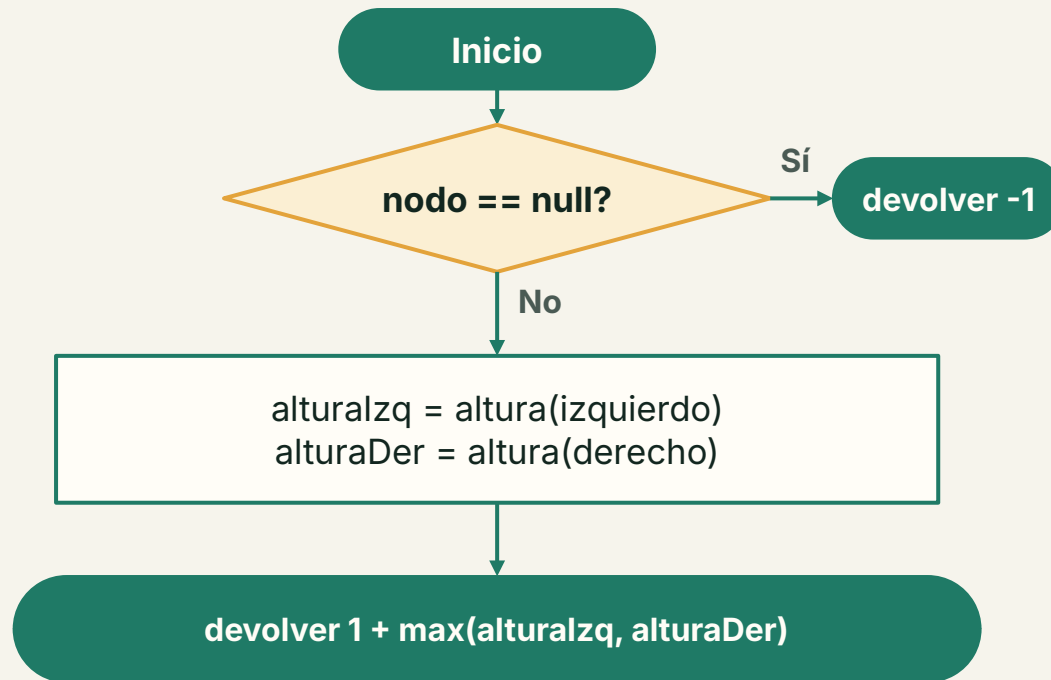
```
int contarHojas(Nodo<T> nodo)
```



OPERACIONES DEL NEGOCIO

altura()

```
int altura(Nodo<T> nodo)
```



¿Por qué -1 para null?

Un árbol vacío mide -1.

Así una hoja mide $1 + \max(-1, -1) = 0$.

La altura cuenta aristas, no nodos.

Ejemplo

$\text{altura}(\text{CUE}) = 0$

$\text{altura}(\text{GYE}) = 1 + \max(0, 0) = 1$

$\text{altura}(\text{UIO}) = 1 + \max(1, 0) = 2$

EN JAVA

El Main usa las operaciones

UIO

```
+-- I: GYE
|   +-- I: CUE
|   `-- D: LOH
`-- D: MEC
    `-- D: ESM
```

```
contarNodos()  -> 6
contarHojas()  -> 3
altura()       -> 2
gye.esHoja()   -> false
gye.grado()    -> 2
```

1

crearRaiz y agregar

construyen el árbol nodo por nodo.

2

VistaArbol

lo dibuja en consola después de cada paso.

3

contarNodos, contarHojas y altura

lo describen con números.

RETO

Reto: ¿árbol sano o árbol en cadena?

1

Construir

10 min

Armen el árbol A con `crearRaiz`, `agregarIzquierdo` y `agregarDerecho`.

2

Consultar

8 min

Impriman la raíz, los nodos, las hojas, la altura y el grado de LTX y SNC.

3

Decidir

12 min

Armen el árbol B y escriban `esCadena(arbol)`: altura igual a nodos menos 1.

REGLAS

Solo se usan `Nodo` y `ArbolBinario`.

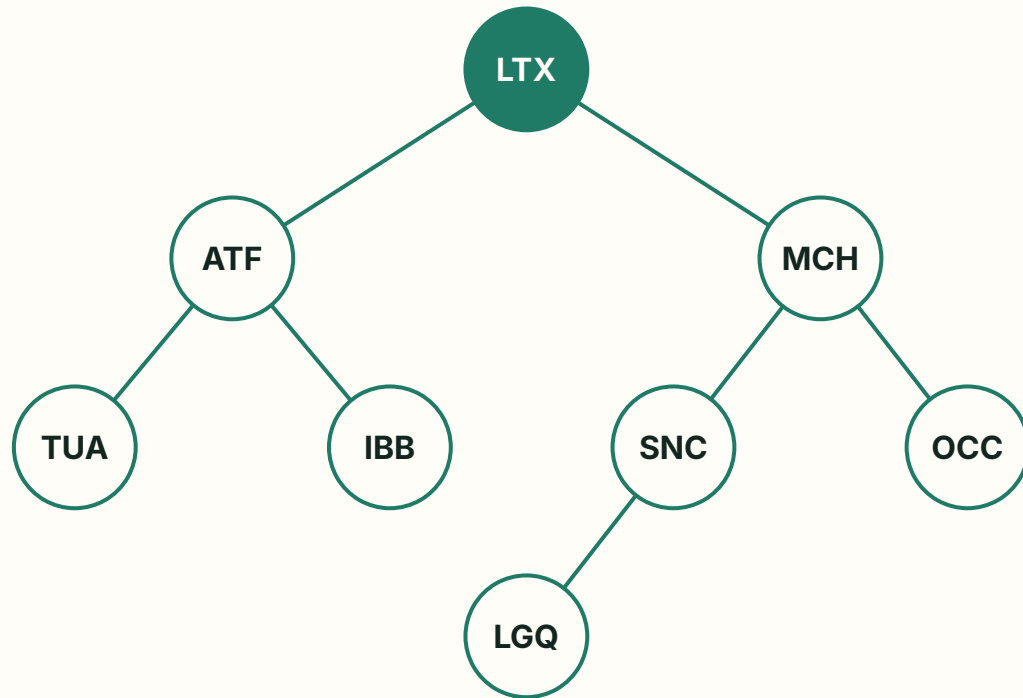
Prohibido: `TreeSet`, `TreeMap`, `PriorityQueue`, `NavigableSet` y `NavigableMap`.

Casos límite: árbol vacío, un solo nodo y agregar donde ya hay un hijo.

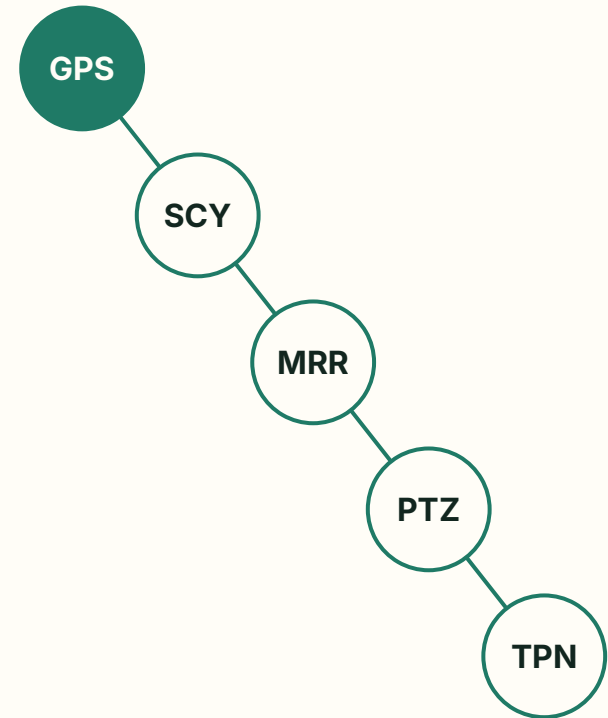
RETO

Los árboles del reto

Árbol A: 8 nodos



Árbol B: 5 nodos



El árbol en nuestro proyecto final

HOY · HITO 1: LISTA

7.698

comparaciones en mi conjunto de datos (aeropuertos) para buscar UKDM, el último de la lista.

13 DE NOVIEMBRE · HITO 4A: ÁRBOL BST

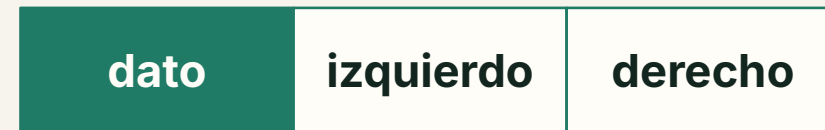
≈ 13

comparaciones como máximo con un árbol equilibrado, usando el ID como clave.

Nodo de la lista: un enlace



Nodo del árbol: dos enlaces



Es la misma idea que ya programamos: un nodo guarda un dato y enlaces. El árbol usa dos enlaces en vez de uno.

CIERRE

Resumen

1

Un árbol organiza datos con jerarquía.

2

La raíz no tiene padre y las hojas no tienen hijos.

3

El árbol binario usa dos enlaces por nodo.

¿Dudas o preguntas?